

# AI FitTrack

Personal Fitness Management System

## PROJECT DOCUMENTATION

### PROJECT TEAM

| Name             | Register No. | Role        |
|------------------|--------------|-------------|
| Somu M P         | 212400292    | Team Leader |
| Vedhanarayanan P | 212400293    | Team Member |
| Naveen Kumar S   | 212400285    | Team Member |
| Perarasu S       | 212400287    | Team Member |
| Pozhilan M       | 212400288    | Team Member |

### TECHNOLOGY OVERVIEW

|                |                                          |
|----------------|------------------------------------------|
| Project Type   | Full-Stack Fitness Management System     |
| Frontend       | HTML + CSS + JavaScript                  |
| Backend        | Node.js + Express.js                     |
| Database       | MongoDB Atlas                            |
| AI Integration | AI-powered fitness recommendation module |

*This document presents the design, implementation, database structure, testing and working features of the AI FitTrack application.*

# 1. Abstract

AI FitTrack is a personal fitness management system designed to help users maintain fitness information in one place. The application combines user authentication, progress tracking, workout management, food information, dashboard statistics and AI-based recommendations.

The frontend provides a simple browser-based interface, while the backend exposes REST APIs for authentication, progress, workouts, food records and dashboard information. MongoDB Atlas is used for persistent storage.

The system reduces manual record keeping and presents fitness information in an organized form. Automatic BMI calculation and dashboard recommendations make the application more useful for everyday fitness tracking.

## 2. Problem Statement

Maintaining fitness information manually can make it difficult to remember workout history, monitor body measurements and review food information. Users may also find it difficult to understand their overall progress when information is stored in separate places.

AI FitTrack addresses these issues by providing a centralized application where authenticated users can record progress, workouts and food details, view historical information and receive summarized dashboard information.

### **Major problems addressed:**

- Scattered fitness records and manual tracking.
- Difficulty reviewing previous workouts and progress.
- No single dashboard for important fitness statistics.
- Need for automatic BMI calculation from weight and height.
- Need for searchable workout and food information.
- Need for personalized recommendation support.

### 3. Objectives

The main objective is to develop a working personal fitness management application with a clear frontend, REST-based backend and MongoDB database.

- Provide secure registration and login.
- Allow users to add and manage progress records.
- Calculate BMI automatically from weight and height.
- Provide progress history with edit and delete operations.
- Allow users to add, search, update and delete workouts.
- Maintain food information with nutrition values.
- Display workout, calorie and food statistics on the dashboard.
- Provide BMI category and recommendation information.
- Provide AI recommendation support.
- Store application data reliably in MongoDB Atlas.

## 4. Scope of the Project

The project is intended for personal fitness tracking. It focuses on recording information, retrieving stored records and presenting useful summaries through a web interface.

### Included in the current system:

- User registration and login.
- JWT-based protected backend operations.
- Progress tracking.
- Automatic BMI calculation.
- Workout CRUD and search.
- Food CRUD and search.
- Dashboard statistics.
- AI recommendation display.
- MongoDB Atlas database storage.

The system is a fitness management aid and does not replace professional medical diagnosis or treatment.

## 5. Software and Hardware Requirements

### Software requirements

- Windows 10/11 or another supported operating system.
- Visual Studio Code.
- Node.js and npm.
- Express.js backend.
- MongoDB Atlas.
- MongoDB/Mongoose libraries.
- A modern web browser.
- Thunder Client for REST API testing.

### Hardware requirements

- A computer capable of running VS Code and Node.js.
- At least 8 GB RAM is recommended.
- Sufficient storage for the project, dependencies and database tools.
- Internet connection for MongoDB Atlas and AI-related services.

## 6. Technology Stack

| Layer          | Technology                | Purpose                                |
|----------------|---------------------------|----------------------------------------|
| Frontend       | HTML, CSS, JavaScript     | User interface and browser interaction |
| Backend        | Node.js, Express.js       | REST API and server-side logic         |
| Database       | MongoDB Atlas             | Persistent application data            |
| ODM            | Mongoose                  | Schema and database operations         |
| Authentication | JWT, bcryptjs             | Login security and password protection |
| Testing        | Thunder Client            | API endpoint testing                   |
| AI             | AI recommendation service | Fitness recommendation support         |

## 7. System Architecture

AI FitTrack follows a layered client–server architecture. The browser-based frontend sends HTTP requests to the Express backend. The backend validates requests, performs business operations and communicates with MongoDB through Mongoose.

Authentication-protected requests include a JWT token. Database operations are performed against MongoDB Atlas. The frontend receives JSON responses and updates the interface accordingly.

### High-level flow

- User → Frontend interface
- Frontend → REST API request
- Express server → Route and controller
- Authentication middleware → Token validation
- Controller/service → Business operation
- Mongoose → MongoDB Atlas
- Backend → JSON response
- Frontend → Display updated result

## 8. Major System Modules

### Authentication Module

Handles registration, login and token storage. Protected functions check the stored token before sending secured requests.

### Progress Module

Stores weight, height, BMI and date. BMI is calculated automatically from weight and height.

### Workout Module

Supports adding, viewing, searching, editing and deleting workout records.

### Food Module

Stores food name, calories, protein, carbohydrates, fats and serving size.

### Dashboard Module

Displays total workouts, calories burned, total foods, latest progress and recommendations.

### Recommendation Module

Displays recommendation information returned by the backend/AI functionality.

## 9. Authentication and Security

The application uses a registration and login flow. During registration, the user supplies a name, email and password. During login, the backend verifies the supplied credentials and returns a token when authentication succeeds.

The frontend stores the received token in `localStorage` and attaches it as a Bearer token to protected requests such as dashboard, progress and workout operations.

- Registration validation.
- Login validation.
- JWT token handling.
- Authorization header for protected API requests.
- Password protection through the backend authentication mechanism.
- Unauthorized access is rejected by the backend.

## 10. Progress Tracking and BMI

The Add Progress module accepts weight, height and date. The BMI field is read-only because the frontend calculates it automatically when weight or height changes.

The BMI calculation converts height from centimetres to metres and divides weight in kilograms by the square of height in metres. The resulting value is shown with two decimal places.

Progress History displays stored records and provides Edit and Delete actions. After a change, the dashboard and progress list are refreshed.

### BMI category display

- Below 18.5 – Underweight
- 18.5 to below 25 – Normal
- 25 to below 30 – Overweight
- 30 and above – Obese

## 11. Workout Management

The workout module allows users to maintain a history of exercise activities. Each workout contains a name, category, duration, calories burned and workout date.

The interface provides Add Workout, Show All Workouts and Search options. Existing records can be edited or deleted. Successful changes refresh both the workout list and dashboard statistics.

- Create workout record.
- View all workouts.
- Search workout records.
- Edit workout information.
- Delete workout information.
- Update dashboard totals after changes.

## 12. Food and Nutrition Records

The food module stores basic nutritional information. A food record contains the food name, calories, protein, carbohydrates, fats and serving size.

Users can add food records and view or search the available food list. Edit and delete operations are also provided in the interface.

- Food name.
- Calories.
- Protein in grams.
- Carbohydrates in grams.
- Fats in grams.
- Serving size.

## 13. Dashboard and Recommendations

The dashboard provides a summary of the user's stored information. After login, the frontend requests dashboard data from the backend and updates the summary cards.

The dashboard includes total workouts, total calories burned and total foods. The latest progress section displays weight, height, BMI and BMI category. Recommendation text is displayed in the AI Recommendation section.

- Total Workouts.
- Calories Burned.
- Total Foods.
- Latest Weight.
- Latest Height.
- Latest BMI.
- BMI Category and recommendation.
- AI Recommendation.

## 14. Database Design – MongoDB Atlas

MongoDB Atlas is used as the cloud database for AI FitTrack. The project uses collections to store structured JSON-like documents.

The working database contains collections for the main application modules. In the current implementation, the Atlas database shows collections including users, workouts, progresses, foods and recommendations.

### Database collections

- users – authentication and user information.
- workouts – workout records.
- progresses – weight, height, BMI and date records.
- foods – nutrition and serving information.
- recommendations – recommendation information.

## 15. Data Relationships

The core user relationship is one-to-many: one authenticated user can have multiple workout and progress records. User-specific records are associated with the authenticated user through the backend data model.

The food and recommendation collections support the nutrition and recommendation portions of the application.

### Representative relationship

- User → many Workouts
- User → many Progress records
- Food → nutrition information
- Recommendation → recommendation information

## 16. REST API Structure

The frontend communicates with the backend using HTTP requests. POST is used for creating records, GET for retrieving data, PUT for updating records and DELETE for removing records.

The frontend code uses a common API base URL and sends JSON request bodies where required. Protected operations include the Authorization header containing the JWT token.

| Operation      | HTTP method         | Purpose                        |
|----------------|---------------------|--------------------------------|
| Register       | POST                | Create user account            |
| Login          | POST                | Authenticate user              |
| Dashboard      | GET                 | Retrieve dashboard information |
| Progress       | GET/POST/PUT/DELETE | Manage progress records        |
| Workouts       | GET/POST/PUT/DELETE | Manage workout records         |
| Workout Search | GET                 | Search workout records         |
| Foods          | GET/POST/PUT/DELETE | Manage food records            |

## 17. Frontend Implementation

The frontend is organized around a single application page containing registration, login, dashboard, progress, workout and food sections. HTML elements are connected to JavaScript functions using IDs and button click handlers.

The stylesheet provides a clean card-based layout, responsive dashboard cards and mobile-friendly sizing. JavaScript handles API requests, form validation, result messages, list rendering and automatic BMI calculation.

### Important frontend functions include:

- registerUser()
- loginUser()
- getDashboard()
- calculateBMI()
- addProgress() / getProgress()
- addWorkout() / getWorkouts() / searchWorkouts()
- addFood() / getFoods() / searchFoods()
- edit and delete functions for stored records.

## 18. Backend Implementation

The backend provides REST endpoints used by the frontend. The server receives JSON data, validates the request, performs the required operation and returns a JSON response.

A modular backend separates authentication, database access, business logic and route handling. MongoDB Atlas is connected through Mongoose.

- Express server configuration.
- CORS configuration for frontend communication.
- MongoDB/Mongoose connection.
- Authentication and authorization.
- Progress, workout and food routes.
- Dashboard aggregation.
- Recommendation handling.
- Error responses.

## 19. Validation and Error Handling

The frontend checks required fields before making requests. The backend should also validate incoming data so that invalid values are not stored.

The application displays result messages for successful operations and connection or request failures.

- Required-field validation.
- Numeric validation for weight, height, duration and calories.
- Authentication checks before protected operations.
- HTTP response status checking.
- Readable success/error messages.
- Console logging for debugging connection failures.

## 20. Testing Methodology

The backend APIs were tested using Thunder Client. Testing was performed after starting the backend server and connecting the application to MongoDB Atlas.

Testing covers authentication, database operations, progress, workouts, foods and dashboard-related requests. The objective is to confirm that the frontend and backend communicate correctly and that records are stored and retrieved as expected.

### Testing sequence

- Check backend root/API response.
- Register a test user.
- Login and verify token response.
- Test protected dashboard request.
- Add and retrieve progress.
- Add, retrieve, search, update and delete workouts.
- Add and retrieve food records.
- Verify MongoDB Atlas collections and stored documents.
- Verify frontend displays the returned data.

## 21. Testing Results

The completed test flow demonstrates that the main application components communicate successfully. The MongoDB Atlas database contains the application collections, and the frontend can display stored information after successful authentication.

Screenshots of the working application and the MongoDB Atlas database are included in the screenshot section of this document.

| Test area      | Expected result                         | Status    |
|----------------|-----------------------------------------|-----------|
| Registration   | User account created                    | Completed |
| Login          | Token returned and dashboard accessible | Completed |
| Progress       | Record stored and BMI displayed         | Completed |
| Workout CRUD   | Create/read/update/delete works         | Completed |
| Workout search | Matching records displayed              | Completed |
| Food records   | Food information displayed              | Completed |
| MongoDB Atlas  | Collections and documents visible       | Completed |
| Dashboard      | Counts and latest progress shown        | Completed |

# Application Screenshots

This section contains screenshots of the working AI FitTrack application, the MongoDB Atlas database and the backend server, arranged in the order in which the features are described.

| No.           | Description                                    | Page |
|---------------|------------------------------------------------|------|
| Screenshot 1  | Frontend – AI FitTrack Main Page .....         | 24   |
| Screenshot 2  | Registration and Login .....                   | 25   |
| Screenshot 3  | Login Success and Dashboard .....              | 26   |
| Screenshot 4  | Add Progress – Automatic BMI .....             | 27   |
| Screenshot 5  | Progress History .....                         | 28   |
| Screenshot 6  | Add Workout .....                              | 29   |
| Screenshot 7  | Workout List and Search .....                  | 30   |
| Screenshot 8  | MongoDB Atlas – FitTrack Collections .....     | 31   |
| Screenshot 9  | Add Food and Food List .....                   | 32   |
| Screenshot 10 | AI Recommendation .....                        | 33   |
| Screenshot 11 | Final Dashboard – Results .....                | 34   |
| Screenshot 12 | Final Application – Complete View .....        | 35   |
| Supplementary | Backend – Server and Database Connection ..... | 36   |

# Project Screenshot 1

## Frontend – AI FitTrack Main Page

### AI FitTrack

Personal Fitness Management System

#### Register

Register

#### Login

Login

#### Dashboard

New Dashboard

Total Workouts

0

Calories Burned

0

Total Foods

0

#### Latest Progress

Weight : N/A  
Height : N/A  
BMI: N/A  
BMI Category: N/A  
BMI Recommendation: No recommendation available.

#### AI Recommendation

No recommendation available.

 **Add Progress**

# Project Screenshot 2

## Registration and Login

### Register

Register

### Login

Login

# Project Screenshot 3

## Login Success and Dashboard

Login

Login

Dashboard

View Dashboard

Total Workouts

6

Calories Burned

1150

Total Foods

4

# Project Screenshot 4

## Add Progress – Automatic BMI

 Add Progress

64

170

22.15

dd-mm-yyyy

Add Progress

## Project Screenshot 5

### Progress History

 Progress History

Show Progress

 Progress

**Weight: 64 kg**  
**Height: 170 cm**  
**BMI: 22.15**  
**Date: 27/9/2026**

 Edit  Delete

 Progress

**Weight: 64 kg**  
**Height: 170 cm**  
**BMI: 22.15**  
**Date: 27/9/2026**

 Edit  Delete

 Progress

**Weight: 64 kg**  
**Height: 170 cm**  
**BMI: 22.15**  
**Date: 27/9/2026**

 Edit  Delete

 Progress

**Weight: 64 kg**  
**Height: 170 cm**  
**BMI: 22.15**  
**Date: 27/9/2026**

 Edit  Delete

 Progress

# Project Screenshot 6

## Add Workout

 **Add Workout**

Add Work out

# Project Screenshot 7

## Workout List and Search

 MyWorkouts

Search

Show AllWorkouts

Running

**Category:** Cardio

**Duration:** 30 minutes

**Calories:** 250

**Date:** 27/9/2026

Edit

Delete

Walking

**Category:** Cardio

**Duration:** 20 minutes

**Calories:** 100

**Date:** 27/9/2026

Edit

Delete

# Project Screenshot 8

## MongoDB Atlas – FitTrack Collections

The screenshot displays the MongoDB Atlas Data Explorer interface. The left sidebar shows the 'Data Explorer' section with 'My Queries' and 'Data Modeling' options. Under 'CLUSTERS (1)', the 'aiifittrack' cluster is selected, showing a tree view of collections: 'foods', 'progresses', 'recommendations', 'users', and 'workouts'. The main panel shows the 'aiifittrack' cluster details, including a table of collections and their properties.

| Collection name | Properties | Storage size | Data size | Documents | Avg. document size | Indexes | Total index size |
|-----------------|------------|--------------|-----------|-----------|--------------------|---------|------------------|
| foods           | -          | 36.86 kB     | 683.00 B  | 4         | 170.00 B           | 1       | 36.86 kB         |
| progresses      | -          | 36.86 kB     | 110 kB    | 8         | 138.00 B           | 1       | 36.86 kB         |
| recommendations | -          | 36.86 kB     | 1.04 kB   | 3         | 348.00 B           | 1       | 36.86 kB         |
| users           | -          | 36.86 kB     | 756.00 B  | 4         | 189.00 B           | 2       | 73.73 kB         |
| workouts        | -          | 36.86 kB     | 1.32 kB   | 7         | 188.00 B           | 1       | 36.86 kB         |

Alert System Status: All Good  
©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

## Project Screenshot 9

### Add Food and Food List

#### Add Food

#### Food List

 Apple

**Calories:** 52  
**Protein:** 0.3 g  
**Carbohydrates:** 14 g  
**Fats:** 0.2 g  
**Serving Size:** 100g

 banana

**Calories:** 52  
**Protein:** 0.2 g  
**Carbohydrates:** 56 g

## Project Screenshot 10

### AI Recommendation

#### Latest Progress

**Weight:** 64 kg  
**Height:** 170 cm  
**BMI:** 22.15  
**BMI Category:** Normal  
**BMI Recommendation:** Your BMI is in the normal range. Continue maintaining balanced nutrition and regular physical activity.

#### AI Recommendation

Your BMI is in the normal range. Maintain a balanced diet and regular exercise. Try to maintain a more consistent workout schedule during the week. Gradually increase your physical activity according to your fitness level.

# Project Screenshot 11

## Final Dashboard – Results

### Login

Login

### Dashboard

View Dashboard

Total Workouts  
**6**

Calories Burned  
**1150**

Total Foods  
**4**

#### Latest Progress

**Weight:** 64 kg  
**Height:** 170 cm  
**BMI:** 22.15  
**BMI Category:** Normal  
**BMI Recommendation:** Your BMI is in the normal range. Continue maintaining balanced nutrition and regular physical activity.

#### AI Recommendation

Your BMI is in the normal range. Maintain a balanced diet and regular exercise. Try to maintain a more consistent workout schedule during the week. Gradually increase your physical activity according to your fitness level.

 **Add Progress**

Add Progress

# Project Screenshot 12

## Final Application – Complete View

AI FitTrack

Personal Fitness Management System

Register

Register

Login

Login

Dashboard

View Dashboard

Total Members

5

Calories Burned

950

Total Weeks

4

Latest Progress

Weight: 85kg

Height: 1.80m

BMI: 26.75

BMI Category: Normal

BMI Recommendation: Your BMI is in the normal range. Continue maintaining a balanced diet and regular physical activity.

AI Recommendation

You should focus on strength training to build muscle and regular aerobic exercise to improve cardiovascular health. Consider incorporating high-intensity interval training (HIIT) 2-3 times per week for additional calorie burn and metabolic boost.

Add Progress

170

2023-10-27

Add Progress

Progress History

Show Progress

Click "Show Progress" to view your progress.

Add Workout

2023-10-27

Add Workout

My Workouts

Search

Show All Workouts

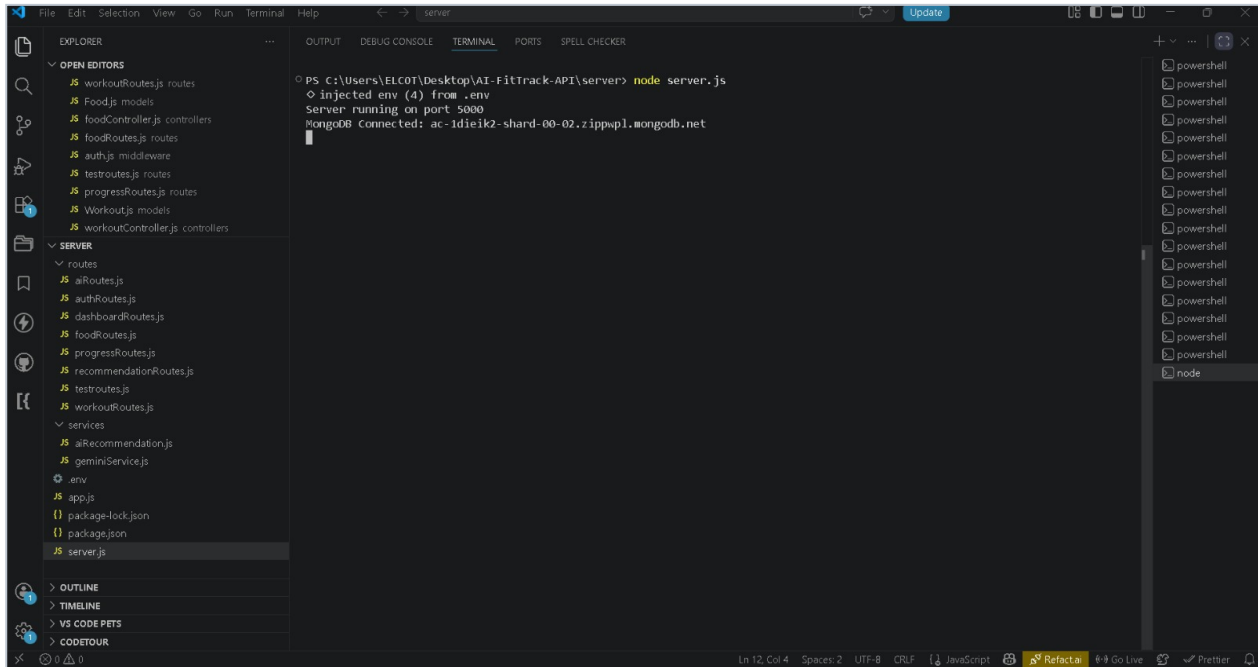
Click "Show All Workouts" to view your workouts.

AI FitTrack Project Documentation | Page 35

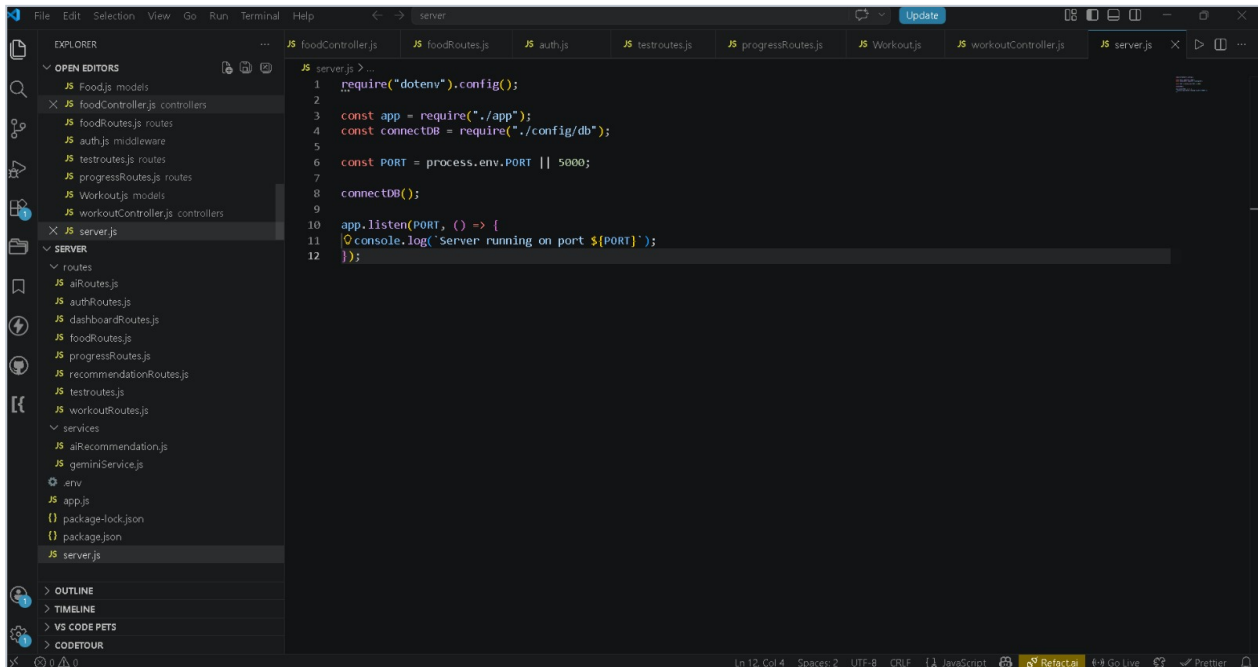
# Supplementary Screenshots

## Backend – Server and Database Connection

### A. Backend server running on port 5000 with MongoDB Atlas connected



### B. server.js – Express server entry point and database connection call



## 22. Conclusion

AI FitTrack successfully combines a browser-based frontend, REST backend and MongoDB Atlas database into a single fitness management system. The application provides authentication, progress tracking, automatic BMI calculation, workout management, food records and dashboard summaries.

The project demonstrates practical use of HTML, CSS, JavaScript, Node.js, Express.js, MongoDB, Mongoose, JWT-based authentication and API testing with Thunder Client. The modular design also provides a foundation for adding more fitness-related functionality in the future.

## 23. Future Enhancements

The current system can be extended without changing its basic architecture.

- Charts for weight and workout progress.
- More detailed nutrition analytics.
- Workout reminders and notifications.
- Wearable-device integration.
- Role-based administration.
- Improved AI-generated plans and progress insights.
- Exporting progress reports.
- Cloud deployment of frontend and backend.
- Additional validation and automated testing.

## 24. Project Documentation Summary

This report has been prepared as a project documentation document for AI FitTrack. It describes the actual application modules, technology stack, database structure, frontend behaviour, backend communication and testing workflow.

Screenshots of the working application, the MongoDB Atlas database and the backend server are included in the screenshot section of this document as evidence of the completed project.

End of Project Documentation